

RK0 Functional Specification Document

Table of Contents

1. Purpose and reading convention	1
2. Real-time and interaction model	2
3. Task execution and scheduling	3
4. Time and waiting contracts	4
5. Signals and shared-state coordination	5
5.1 Retained tokens	5
5.2 Ownership and pure waiting	5
6. Message contracts	6
6.1 Completion and retention	6
6.2 Buffered queues and mailboxes	6
6.3 Asynchronous direct messages	7
6.4 Rendezvous and invocation	7
6.5 Most-Recent Message publication	8
7. Combined priority contract	9
8. Storage, execution context and diagnostics	9
9. Contract traceability and evolution	10
Source register	11

1. Purpose and reading convention

REV:11SEP26/0801 | © 2026 www.antoniogiacomelli.com

This specification defines the externally meaningful behaviour of RK0: what makes a task eligible to run, what each interaction retains, when an operation completes, and how time, ownership and priority affect that interaction. It is intended for application designers, kernel maintainers and reviewers assessing a proposed change.

RK0 is an interaction-oriented real-time executive for a single-core embedded system. Tasks, application code and kernel code form one linked image in a flat address space. Tasks execute privileged. The application supplies the resource capacities, task priorities, execution bounds and coordination policy needed to meet its timing requirements. [D1, K1, K2]

Contract identifiers such as **F-SCH-01** are stable references shared with the architecture specification. ``Shall'' states the contract of this software baseline; it does not indicate a separately approved future requirement. Application obligations are stated explicitly. API identifiers retain their source spelling.

The baseline sources are the current DocBook **0801.0.0.1**, the service and requirements wiki, the design blog, and the implementation. Exact implementation behaviour governs the baseline where explanatory prose differs. The source register records immutable repository revisions. [D1, W1, W2, B1]

2. Real-time and interaction model

An event is an occurrence associated with a change of system state. Its representation determines what the software can observe later. RK0 distinguishes a pure signal, a retained signal token and a message. These convey different coordination contracts even when their C representations are small. [D1, B1]

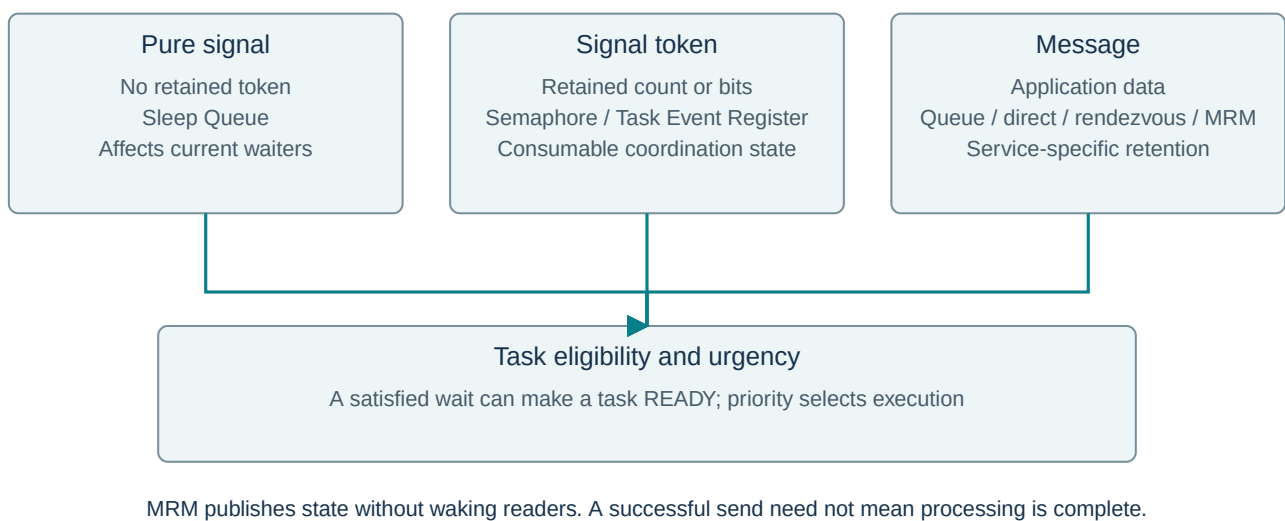


Figure 1. Figure F1. Representations of an occurrence and their effect on execution eligibility.

Concept	Meaning in RK0
Task	A sequential execution context with a stack, saved machine context, priority and service state.
READY	Eligible for selection by the scheduler. It does not mean execution has started.
WAITING	Conceptual family of service-specific suspensions. A wait names a release condition.
Pure signal	An operation affecting current waiters without retaining a token; a Sleep Queue signal is the example.
Signal token	Consumable state retained as a semaphore count or Task Event Register bits.
Message	Application data with an explicit retention, transfer, ownership and completion rule.
Nominal / effective priority	Assigned scheduling priority / priority currently used after interaction contributions. Lower numbers express greater urgency.

F-MOD-01 — Explicit progress. Execution progress shall follow task eligibility, priority, explicit yielding and service completion. A periodic tick alone shall not rotate equal-priority tasks. Application code expresses where a cycle ends or where further progress depends on something else.

F-MOD-02 — Semantic preservation. Replacing one service with a composition is equivalent only if it preserves the relevant completion point, retained information, ownership, waiting, timeout and priority effects. Delivering the same bytes eventually is insufficient. This criterion also guides the addition of new services. [B1, B2]

MRM and condition waiting illustrate this design choice. Most-recent publication has a stable 1:N contract with nontrivial buffer lifetimes, so RK0 implements it as a service. A condition variable depends on an application-owned predicate and signalling policy, so RK0 exposes Mutex and Sleep Queue objects with composition helpers. Neither a wake-up nor a signal proves an application predicate true.

3. Task execution and scheduling

The normal policy is fixed-priority preemptive scheduling with FIFO order within each priority. The baseline supports normal priorities **0–31**; priority 0 is highest. Idle is a fallback below that range. The kernel also runs a post-processing system task. [D1, K2]

Contract	Required behaviour
F-SCH-01 Selection	At a permitted dispatch, select the highest-priority READY task. The scheduler and architecture port together perform READY → RUNNING.
F-SCH-02 Equal priority	A task preempted by more urgent work returns to the head of its ready queue. A yielding task enters the tail when an eligible peer exists. A released waiter normally enters the tail. Initial peers are ordered by creation.
F-SCH-03 Yield	kYield() does not grant CPU time to lower-priority work merely because the current task yielded. Equal-priority tasks must yield or wait to share execution.
F-SCH-04 Priority consistency	An effective-priority change shall update READY placement or an associated priority-ordered wait queue. A RUNNING task's change shall trigger reconsideration of scheduling.
F-SCH-05 Scheduler lock	kSchLock() nests per task. Preemption is deferred until the outermost unlock. If the task blocks, another task runs with its own lock state; the blocked task's depth is restored on resumption. Interrupts remain possible.

RK_NO_PREEMPT is an exceptional task-creation option. Such a task can delay other tasks until it yields or waits; its execution must therefore be included in the application's blocking analysis. Fixed-priority scheduling does not by itself enforce deadlines, fairness or CPU budgets.

Lifecycle stage	Observable meaning and permitted progression
INITIALISED	Task context exists. Startup or creation makes it READY.
READY → RUNNING	Dispatch, preemption and explicit yield determine progress.
RUNNING → WAITING → READY	A service suspends the task; satisfaction of its condition or expiry of its wait makes it eligible again.
PENDING → TERMINATED	Deferred reclamation of an eligible runtime-spawned task. This is a lifecycle path, not a service wait.

F-LIFE-01 — Task storage. `kTaskInit()` uses an application-provided stack and a kernel TCB slot. `kTaskSpawn()` additionally obtains its stack from a configured fixed-block partition. Task capacities remain bounded. Stack sizes are expressed in words; stack storage must satisfy the port's alignment and minimum size requirements.

F-LIFE-02 — Termination. Only runtime-spawned application tasks are terminable. Termination is rejected while protected dependencies remain, including mutex ownership/waiting and outstanding direct-message relationships. The accepted state set is limited: READY, time/event sleeps and the deferred PENDING path. Object waits, including `RK_WAITING_ALLOC`, are not a general cancellation interface. Self-termination defers storage reclamation so the running stack is not freed beneath its user. [K2]

4. Time and waiting contracts

Time is expressed in ticks. A finite interval must lie within `RK_MAX_PERIOD` (0x7FFFFFFF on these 32-bit ports). `RK_NO_WAIT` is zero; `RK_WAIT_FOREVER` is the distinct 0xFFFFFFFF sentinel. Millisecond conversion follows the configured tick granularity. [D1, K3]

Contract / service	Completion and overrun behaviour
F-TIME-01 <code>kBusyDelay()</code> / <code>kDelay()</code>	Does not suspend the task. It decrements its work counter once per observed tick change; elapsed ticks during preemption are not accumulated. This is a tick-granular workload approximation, not an exact CPU-cycle budget.
F-TIME-02 <code>kSleepDelay()</code> / <code>kSleep()</code>	Suspends relative to the call. Computation and dispatch lateness add to successive cycle lengths. Zero delay returns <code>RK_ERR_TIMEOUT</code> ; it is not a yield.
F-TIME-03 <code>kSleepUntil()</code>	Advances the caller-supplied anchor by one period. If the target is already reached, returns <code>RK_ERR_ELAPSED_PERIOD</code> without sleeping and increments the overrun counter. Repeated calls can account for missed activations individually.
F-TIME-04 <code>kSleepRelease()</code>	Uses the TCB release anchor, initially the scheduler-start grid. It selects the next phase strictly after the current time, skipping elapsed releases. The overrun counter increments once per late call, not once per skipped release.

Contract / service	Completion and overrun behaviour
F-TIME-05 Callout timer	First expiry is <code>phase + period</code> after initialisation. One-shot timers fire once; reload timers retain their phase and skip elapsed release points. Callbacks execute in the post-processing task.

F-WAIT-01 — Expiry and return. A finite timeout bounds an armed suspension. Expiry makes the task READY with a timeout outcome; it does not guarantee that the API has returned at that wall-clock instant. Dispatch interference and any subsequent work still apply. Some retrying service paths can arm another wait; an end-to-end deadline must be treated separately.

F-WAIT-02 — No-wait exceptions. Most waiting APIs use `RK_NO_WAIT` as a try operation. `kSynchMesgCall()` and `kCondVarWait()` reject it. `kSleepQueueSleep(..., RK_NO_WAIT)` returns `RK_ERR_NOWAIT` without enqueueing. The exact service contract takes precedence over a generic timeout convention.

The application shall distinguish release time from execution time and provide bounded callback work, interrupt interference, critical sections and storage demand. RK0 provides the execution rules on which response-time analysis can be based; it does not perform schedulability admission.

5. Signals and shared-state coordination

5.1 Retained tokens

F-SIG-01 — Semaphores. A successful pend consumes one available unit. If none is available, the task can try, wait finitely or wait indefinitely. Post releases the highest-priority waiter before increasing the stored count. With no waiters, binary semaphores saturate at one and counting semaphores saturate at their configured maximum. Semaphores have no owner and do not provide mutex priority inheritance. [K4]

F-SIG-02 — Task Event Register. Each task has a private 32-bit occurrence register. `kEventSet()` ORs bits into it. `kEventGet()` waits for ANY or ALL requested bits and consumes the requested mask on successful completion. Repeated settings of an already set bit coalesce. The optional returned value is a snapshot of the register, so consumers mask it with the requested bits when necessary. Setting a bit carries no payload and does not count repeated occurrences. [K4]

5.2 Ownership and pure waiting

F-MUT-01 — Mutex ownership. A mutex has one owner. Unlock by a non-owner and recursive locking of the same mutex are invalid. Nested acquisition of different mutexes is supported. The selectable protocols are no inheritance and priority inheritance; mutex PCP is not implemented. [K4]

F-MUT-02 — Inheritance. An inherited-priority mutex raises its owner to the urgency required by its waiters. Inheritance propagates through nested mutex wait chains. Unlock, handover and waiter timeout remove obsolete contributions while preserving those from other owned resources. Lock ordering remains an application obligation; inheritance does not resolve circular waiting.

F-SLPQ-01 — Sleep Queue. A Sleep Queue stores waiters, not signals or a predicate. Signal wakes the highest-priority waiter; wake can release a bounded number or all current waiters. A signal to an empty queue retains nothing for a future waiter. Explicit ready/unready operations also support suspending an already READY task on a Sleep Queue. [K4]

F-CV-01 — Condition composition. The application holds a mutex while testing its predicate. `kCondVarWait()` releases that mutex and enters the Sleep Queue without an intervening task dispatch; on wake it reacquires the mutex before normal return. The application retests its predicate in a loop.

In this baseline the finite timeout covers the condition wait; mutex reacquisition uses `RK_WAIT_FOREVER`. Consequently it is not a bound on the complete helper call. A condition-wait timeout also invokes `K_PANIC` after reacquisition. This behaviour must be considered when choosing the helper for a recoverable timeout policy. [K4]

6. Message contracts

6.1 Completion and retention

Service	Success means	Retained information
Message Queue	The value was copied into storage or directly to a waiting receiver.	Bounded FIFO history, except explicit jam/overwrite operations.
Single-slot overwrite	The new value became the pending value or was directly delivered.	One pending value; earlier pending data can be replaced.
Plain rendezvous	The receiver copied the offered payload.	Sender-owned offer while pending; no independent payload backlog.
Extended rendezvous	The server committed a reply to the caller.	Pending calls plus one accepted call per server.
Asynchronous direct message	An allocated message was transferred to the destination endpoint.	Pool-backed message objects, queued or received.
MRM	A complete version became current.	Most-recent state and older versions still held by readers.

The services support different application constructions. Shared state under a mutex, an Active Object with an inbox, and a synchronous client/server service all use ordinary RK0 tasks. They do not introduce another schedulable entity. [D1, B1, B3]

6.2 Buffered queues and mailboxes

F-MSGQ-01 — Transfer. Message Queues copy fixed-size values of 1, 2, 4 or 8 words. Each queue fixes its message size and capacity at initialisation. Normal send/receive retain FIFO message order. Waiter order is a separate priority discipline. A full send or empty receive can wait or report

immediate unavailability. [K5]

F-MSGQ-02 — Variants. Jam inserts at the front. Peek reads without consumption. A single-slot queue provides a mailbox; overwrite replaces its pending value. Broadcast targets the current broadcast receivers and holds the slot until that cohort completes. Normal consumption or overwrite does not steal that in-progress broadcast. With no eligible broadcast waiters, the operation reports an empty outcome and does not retain a new broadcast for future arrivals.

F-MSGQ-03 — Notification and ownership. A configured send callback executes synchronously within the applicable send operation, including its critical section. It must be short and non-blocking. Overwrite does not invoke this callback. A queue containing pointers copies only pointer values; allocation, payload lifetime and ownership then belong to the application. A queue associated by convention with one server is a useful pattern, but the current queue object has no kernel-enforced owner or automatic server boost.

6.3 Asynchronous direct messages

F-ASY-01 — Endpoint and ownership. `kMesgEndpointInit()` adds an inbox to a task. `kMesgPoolInit()` defines a fixed-capacity pool. Allocation gives a task a message; `kMesgSend()` transfers it to the receiver at send time. The receiver owns it while queued and after receipt. Payloads are passed by reference without a send-time payload copy. After transfer, the sender shall stop using the message. [K6]

F-ASY-02 — Admission and waiting. `kMesgSend()` has no timeout argument and does not wait for a receiver or for endpoint capacity. Resource admission occurs in `kMesgAlloc()`, which supports no-wait, finite and indefinite waiting. `kMesgWait()` receives from any sender or a selected sender, preserving nonmatching messages. A freed block can pass directly to the highest-priority waiting allocator.

A strictly asynchronous Active Object front end uses **no-wait allocation followed by send**. Exhaustion then returns immediately; the application decides whether to discard the request, report overload or use another explicit policy. Blocking allocation is a separate, supported resource-wait contract. Successful submission does not acknowledge processing. [B3]

F-ASY-03 — Pool ceiling. An optional fixed priority ceiling contributes while a task owns a message from that pool or waits to allocate one. Ownership moves that contribution from sender to receiver. Freeing the final relevant message, handing it to another allocator or leaving the allocation wait removes that contribution. Other active priority contributions remain.

Admission is checked against effective priority: a task already more urgent than the pool ceiling cannot acquire ownership through allocation or as the send destination. Choose the ceiling to cover intended participants. This is a per-pool ownership rule, not mutex PCP or a system-wide ceiling. ISR allocation is permitted only as no-wait on a ceiling-disabled pool; direct send and receive are task operations. [K6]

6.4 Rendezvous and invocation

F-SYNC-01 — Plain rendezvous. Initialise the receiving task's synchronous endpoint and

maximum message size. `kSynchSendWait()` offers word-sized data; `kSynchRecv()` copies it. A pending sender remains blocked with its source storage valid until receipt or timeout. Receiver priority can be raised while senders wait. Successful send means the copy completed, not that the receiver finished processing. [K7]

F-SYNC-02 — Call, accept, reply. `kSynchMesgCall()` queues a request and keeps the caller blocked through reply. `kSynchMesgAccept()` copies the selected request into server storage and records one active call. `kSynchMesgReply()` copies the reply, completes the transaction and makes the caller READY. The server has no additional wait for caller acknowledgement; ordinary preemption may nevertheless run the caller before Reply returns.

F-SYNC-03 — Priority adoption. Before acceptance, queued callers can boost the server. On acceptance, the caller's effective priority is captured as the server's scheduling base, replacing the server's nominal priority. This can raise or lower the server. More urgent queued callers, direct senders or other applicable contributions can still raise its effective priority. A later caller does not replace the accepted transaction. Reply removes the adopted base and recomputes the remaining contributions. [K2, K7]

F-SYNC-04 — Timeout. The call's timeout continues across queueing and acceptance. Before acceptance it removes the request. After acceptance it marks the call ABANDONED, removes its adopted-priority contribution and prevents subsequent reply writes into the caller's buffers. The server closes the abandoned transaction with Reply; until closure it cannot accept another call, and the caller cannot start another outbound synchronous transaction. Timeout does not undo application work already performed.

The synchronous APIs enforce endpoint size and transaction-state constraints. Request sizes and reply capacity are nonzero word multiples; an actual reply may be empty. The relevant send/receive/call/accept/reply operations reject a task that owns mutexes. Do not retain and use the exposed reply-buffer pointer after the caller has timed out; use Reply for completion. [K7]

6.5 Most-Recent Message publication

F-MRM-01 — Freshness and independence. MRM supports one publisher and independently paced readers. Publish does not notify or wake readers. A reader can observe the same state repeatedly or skip intermediate publications. The service protects complete copies and buffer lifetimes rather than preserving message history. [D1, K8]

F-MRM-02 — Buffer lifecycle. Reserve obtains a writer buffer; Publish copies the payload and commits it as current. Get pins a current version and copies it to the caller's destination; its returned token must be paired with Unget. An obsolete buffer is reclaimed after its last reader releases it. Under the conventional one-outstanding-token-per-reader discipline, provision **N readers + 2 buffers**; other retention patterns require their own capacity bound.

F-MRM-03 — Availability. Reserve and Get are non-blocking and can return null. Get does so before first publication and also while the current buffer is reserved for reuse. Reserve may reuse the current version when it has no readers. Thus data integrity is guaranteed by the protocol, but continuously available Get success is not promised across the reserve/publish interval. [K8]

7. Combined priority contract

For a task T , let **base**(T) be its nominal priority, or the captured active caller priority while serving an ACTIVE extended rendezvous. Its effective priority is the numerically smallest applicable value:

effective(T) = **min**(**base**(T), **mutex waiters**, **queued direct senders**, **queued callers**, **owned-message ceilings**, **allocation-wait ceiling**).

Empty contribution sets add nothing. An abandoned call adds no adopted base. This formulation distinguishes adoption, which replaces the base, from boosting, which can only increase urgency. [K2]

F-PRIO-01 — Composition and removal. Adding or removing a dependency shall recompute effective priority from the remaining sources, rather than restoring a saved nominal value unconditionally. Both rising and falling priority changes must preserve scheduler and wait-queue order.

The automatic cross-task propagation helper follows **mutex inheritance chains**. The formula does not imply a general fixed-point solver over every possible mixture of message dependencies. In particular, active-call adoption is captured at accept; it is not a continuously sampled alias of the caller's priority. Service transitions explicitly update their affected endpoints. These boundaries matter when introducing new compositions.

Example: a server with nominal priority 2 accepts a caller at 7 and can run with base 7. A queued caller at 3 raises it to 3 while the first request remains active. On reply, base 2 returns; the next accept may establish a different base. Independent contributions are retained throughout.

8. Storage, execution context and diagnostics

F-MEM-01 — Bounded allocation. A memory partition manages homogeneous blocks from supplied storage. Allocation returns a block or null without waiting or heap growth. Dynamic service objects draw control blocks from configured per-type pools after `kObjPartitionsInit()`. Supporting queue buffers, MRM storage and dynamic stack partitions still require explicit provisioning. [K9]

F-MEM-02 — Destruction and lifetime. Destroy accepts only dynamically created objects from the proper pool, checks service-specific live-use conditions and clears the passed handle on success. It is not a general flush-and-cancel operation. Application-held aliases must not outlive destruction. The flat model provides neither MPU isolation nor capability-style handle revocation.

F-CTX-01 — Interrupts and callbacks. An ISR shall not perform a blocking wait. `RK_NO_WAIT` alone is not permission to invoke every API from an ISR. Use the documented ISR-compatible subset, such as event set and the supported semaphore/queue operations. Some multi-waiter operations defer work to the system task; retained object pointers must remain valid until that work finishes. Callouts shall complete promptly without blocking that system task. [K1, K10]

F-DIAG-01 — Outcomes. Treat unavailable resources, elapsed periods and timeouts as defined outcomes where the service specifies them. Invalid objects, ownership violations and illegal contexts are programming faults. Error checking, fault handling, assertions and trace are build-configured; disabling checks does not relax valid-use preconditions. Callers should test named return values rather than treating every nonzero value as an identical failure. [K1, K11]

F-DIAG-02 — Observability. Version queries identify the compiled kernel. Optional trace reports tasks, service objects, effective-priority changes and overruns within configured storage limits. WAITING pseudo-states express the reason for suspension; **RK_WAITING_ALLOC** is reported as **ALLOC**. Stack painting and optional switch-time sentinel checks detect some corruption; they do not protect memory. [K11]

9. Contract traceability and evolution

Contract group	Architecture section	Existing evidence location
F-MOD, F-SCH	ASD §§2, 4	DocBook execution model; scheduler source; wiki SCH_RQ family.
F-LIFE, F-MEM	ASD §§3, 9	Task and dynamic-object implementation; wiki lifecycle and MEM_RQ families.
F-TIME, F-WAIT	ASD §§5, 8	Timer implementation; wiki SLPT_RQ and ATMR_RQ families.
F-SIG, F-MUT, F-SLPQ, F-CV	ASD §§5, 6	Service implementations; wiki SEMA_RQ, MUTX_RQ, TEVT_RQ and SLPQ_RQ families.
F-MSGQ, F-ASY, F-SYNC, F-MRM	ASD §7	Communication implementations; wiki message/MRM families; public examples 08 and 10.
F-PRIO	ASD §6	Public examples 06–10 and effective-priority implementation.
F-CTX, F-DIAG	ASD §§8–10	Architecture port, post-processing, trace and fault implementation.

These are evidence pointers, not a claim that each paragraph has an executable test. The wiki’s test mapping and the public CI workflow have different visibility: parts of the harness are private. The architecture specification identifies the public regression targets. [W2, K12]

When a service evolves, retain its contract ID and update the affected completion, retention, ownership, priority and timeout rules together with the implementation mapping. Add a focused regression when the externally observable contract changes. A new status value also requires review of wake-up, termination eligibility and diagnostics.

Source register

Code references K1–K12 resolve to commit [77bf7b1906edf1dfd8391816dcfac99b4d3885c4](#). DocBook and wiki revisions are recorded separately. Blog pages were consulted on 12 September 2026.

D1 — [RK0 DocBook 0801.0.0.1](#), system 0.80.1; docs revision d13a608e03e4.

K1 — Public surface and configuration: [kapi.h](#), [kcommondefs.h](#), [kconfig.h](#).

K2 — Scheduler and task representation: [ksch.c](#); [kobjs.h](#).

K3 — Time and waits: [ktimer.c](#); [ktimer.h](#).

K4 — Shared-state services: [ksema.c](#), [kmutex.c](#), [ksleepq.c](#), [ktaskevents.c](#).

K5 — Copied communication: [kmesgq.c](#); [kringbuf.c](#).

K6 — Asynchronous direct messages: [kmesg.c](#).

K7 — Synchronous rendezvous and invocation: [ksynchmesg.c](#).

K8 — Most-recent publication: [kmrm.c](#).

K9 — Allocation and dynamic objects: [kmem.c](#); [kdynobjs.c](#).

K10 — Machine layer and system work: [ARMv6-M](#); [ARMv7-M](#); [ksystasks.c](#); [syscalls.c](#).

K11 — Diagnostics and identity: [ktrace.c](#), [kerr.c](#), [kversion.c](#).

K12 — Build and regression evidence: [Makefile](#); [CI workflow](#); [public examples](#); [README.md](#).

W1 — [Service Semantics](#), wiki snapshot 85f43fb49554.

W2 — [Requirements Mapping](#), wiki snapshot 85f43fb49554. Test names are traceability references.

B1 — [If Everything Is an Event, Nothing Is](#).

B2 — [RK0: a Software Engineering Perspective](#).

B3 — [Active Objects in RK0](#).